MSDS 451 Programming Assignment 1

Predicting Next-Day Direction of Palantir (PLTR) Returns with Engineered Lag Features and XGBoost

Nadezhda Shiroglazova | Northwestern University, MSDS 451 | April 2026

1. Problem Description

This project replicates the methodology from the WTI jump-start example on a different asset of personal interest: Palantir Technologies (ticker PLTR). Palantir is a U.S. data-analytics and AI-platform company that has become one of the most-watched names in the post-2023 generative-AI cycle. Its share price has been volatile and frequently driven by news flow, which makes it an interesting candidate for short-horizon directional forecasting: if any structure exists in lagged price/volume features, an asset like PLTR is plausibly where it would show up.

The forecasting task is binary classification: given trading-day t, predict whether the log return on day t+1 will be strictly positive (Target = 1) or zero/negative (Target = 0). The motivation is twofold:

- (a) Pedagogical — practice the same feature-engineering, subset-selection, time-series cross-validation, and gradient-boosted-tree workflow demonstrated for WTI.
- (b) Personal — gauge whether a small set of price/volume features carries any directional signal for a stock the author is considering buying.

The technical-report standard from Miller (2025) sets the methodology: engineer 15 lag/EMA-based features, run all-possible-subsets logistic regression with AIC for feature selection, train an XGBoost classifier under TimeSeriesSplit cross-validation, tune via randomized search, and evaluate with ROC AUC, a confusion matrix, and a classification report.

2. Data Preparation and Pipeline

Daily OHLCV data for PLTR were retrieved from Yahoo! Finance via the yfinance package. The download window starts at the IPO date (2020-09-30) and runs through 2026-04-19, yielding 1,393 raw trading days. After computing the lag and EMA warm-up rows and dropping nulls, 1,390 rows remain for modeling. Compared with WTI's 5,000+ training rows, the PLTR sample is roughly four times smaller — a meaningful limitation that informs how much we can trust any out-of-sample result.

All feature engineering and DataFrame operations use Polars. The engineered feature set mirrors the jump-start specification exactly so that results are directly comparable:

- Lagged closing prices: CloseLag1, CloseLag2, CloseLag3.

- Daily intraday range and its lags: HML = High − Low, with HMLLag1, HMLLag2, HMLLag3.
- Daily open-to-close move and its lags: OMC = Open − Close, with OMCLag1, OMCLag2, OMCLag3.
- Lagged volume: VolumeLag1, VolumeLag2, VolumeLag3.
- Exponential moving averages of CloseLag1 (i.e. computed from the previous day's close to avoid leakage): CloseEMA2, CloseEMA4, CloseEMA8 — half-lives of 1, 2, and 4 days.

The continuous response is the daily log return, LogReturn = ln(Close_t / Close_{t-1}). The binary classification target is Target = 1 when LogReturn > 0 and Target = 0 otherwise.

Class balance was checked before modeling: 704 up days (50.6%) and 686 even/down days (49.4%). The classes are essentially balanced, so no resampling was applied. All 15 features were standardized with scikit-learn's StandardScaler before being passed to either the logistic regression used in subset selection or the XGBoost classifier used downstream.

3. Research Design

3.1 Feature selection — all possible subsets with AIC

Following Miller (2025), feature selection was performed by exhaustive enumeration: every non-empty subset of the 15 candidate features (2^15 − 1 = 32,767 subsets) was fit to a logistic regression on the binary target, and the Akaike Information Criterion was computed as AIC = 2k − 2 ln L, where k is the number of model parameters (features + intercept) and L is the maximized likelihood. Lower AIC is preferred — it rewards goodness of fit and penalizes complexity.

The ten lowest-AIC subsets were collected, and the features that appeared most often across those ten subsets were retained for downstream modeling. This recovers a small, robust feature subset without committing to any single model's idiosyncrasies.

3.2 Cross-validation design

Because trading-day observations are temporally dependent, ordinary k-fold cross-validation would leak future information into training folds. Scikit-learn's TimeSeriesSplit was used instead, with n_splits = 5 and a 10-day gap between each training fold and the following test fold. The gap absorbs the longest EMA half-life (CloseEMA8, half-life of 4 days) plus a buffer, so test-set features cannot quietly carry information from training-set days.

3.3 Modeling algorithm and hyperparameter search

XGBoost (XGBClassifier with binary:logistic objective) was the chosen estimator. A baseline run with n_estimators = 1,000 and otherwise default hyperparameters was evaluated under TimeSeriesSplit to establish a starting point. Then a RandomizedSearchCV with 100 iterations sampled from the same five-hyperparameter space used in the jump-start (max_depth, min_child_weight, subsample, learning_rate, n_estimators) under the same TimeSeriesSplit. The

model returned by RandomizedSearchCV was refit on the full sample and used for the final evaluation.

4. Results

4.1 Selected feature subset

The ten lowest-AIC subsets were dominated by intraday-range and intraday-direction features. Frequency counts across the top ten:

- HMLLag2 — appeared in 10 of 10 subsets
- HMLLag1 — appeared in 8 of 10
- OMCLag3 — appeared in 4 of 10
- VolumeLag2 — appeared in 4 of 10
- OMCLag1 — appeared in 3 of 10

These five features were carried forward. This is a striking departure from the WTI result reported in the jump-start, where the selected subset was {CloseLag3, HMLLag1, OMCLag2, OMCLag3, CloseEMA8} — i.e., anchored by lagged closes and the long EMA. For PLTR, none of CloseLag1–3 nor any of the three EMAs survived selection. The correlation heat map (Figure 1) suggests why: HMLLag1, HMLLag2, and CloseLag1 are correlated at $r \approx 0.80$, so the intraday-range features absorb most of the autoregressive information that lagged closes would have contributed, and AIC's complexity penalty then prefers the smaller HML-anchored set. Substantively, this fits PLTR's character as a high-volatility, news-driven name where the daily true range carries more information than the price level itself.

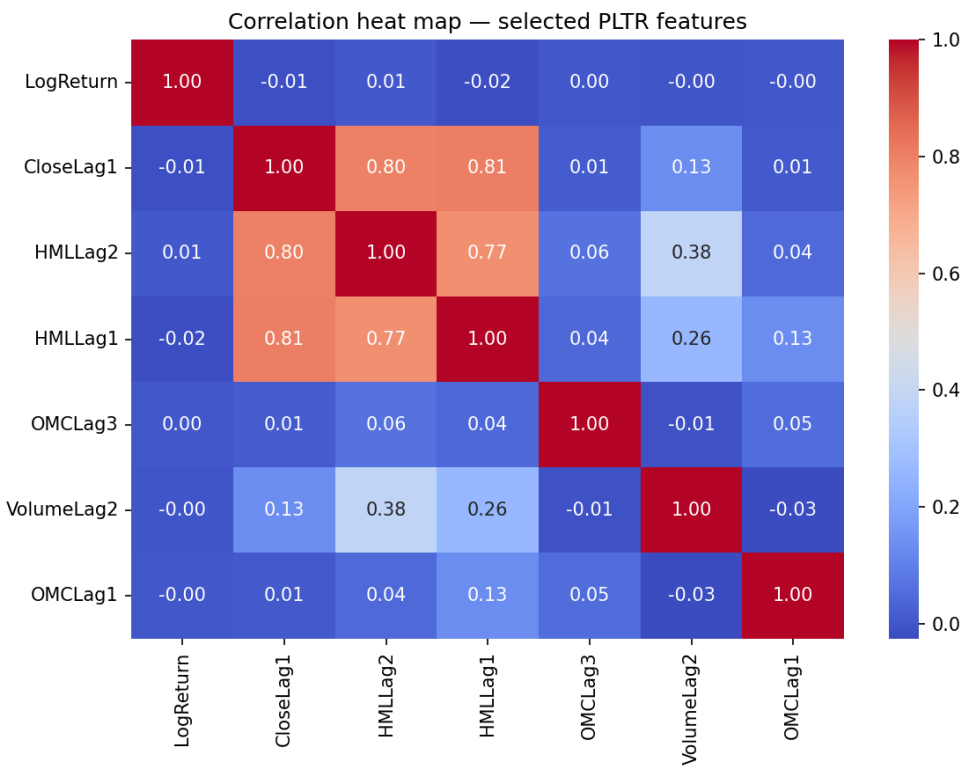

Figure 1. Correlation heat map for the five selected features, plus LogReturn and CloseLag1.

As in the WTI case, every selected feature has a near-zero correlation with LogReturn ($|r| \leq 0.02$). This forecasts up front that the downstream classifier will struggle on out-of-sample test folds, even if it can fit the training data nearly perfectly.

4.2 Cross-validated baseline

The baseline XGBoost classifier (n_estimators = 1,000, defaults elsewhere) achieved a five-fold time-series CV accuracy of 0.506 ± 0.036 — essentially indistinguishable from the 50.6% rate of the majority class. This is the honest signal in the data: with only five lagged features and ~1,400 days of history, the model has approximately no edge over a coin flip on truly held-out days.

4.3 Hyperparameter tuning

RandomizedSearchCV (100 iterations, TimeSeriesSplit gap = 10, n_splits = 5) returned the following best settings:

- max_depth: 6
- min_child_weight: 1
- subsample: 0.893
- learning_rate: 0.014
- n_estimators: 846

Best cross-validated accuracy at these settings was 0.521 — a small improvement over the baseline, but still well within the noise band.

4.4 Final-model evaluation on the full sample

Following the jump-start convention, the final model was refit on the full sample with the tuned hyperparameters and evaluated against the training labels. In-sample ROC AUC was 0.997 (Figure 2). The confusion matrix (Figure 3) shows 669 true negatives, 683 true positives, 17 false positives, and 21 false negatives — overall in-sample accuracy of 97.3%.

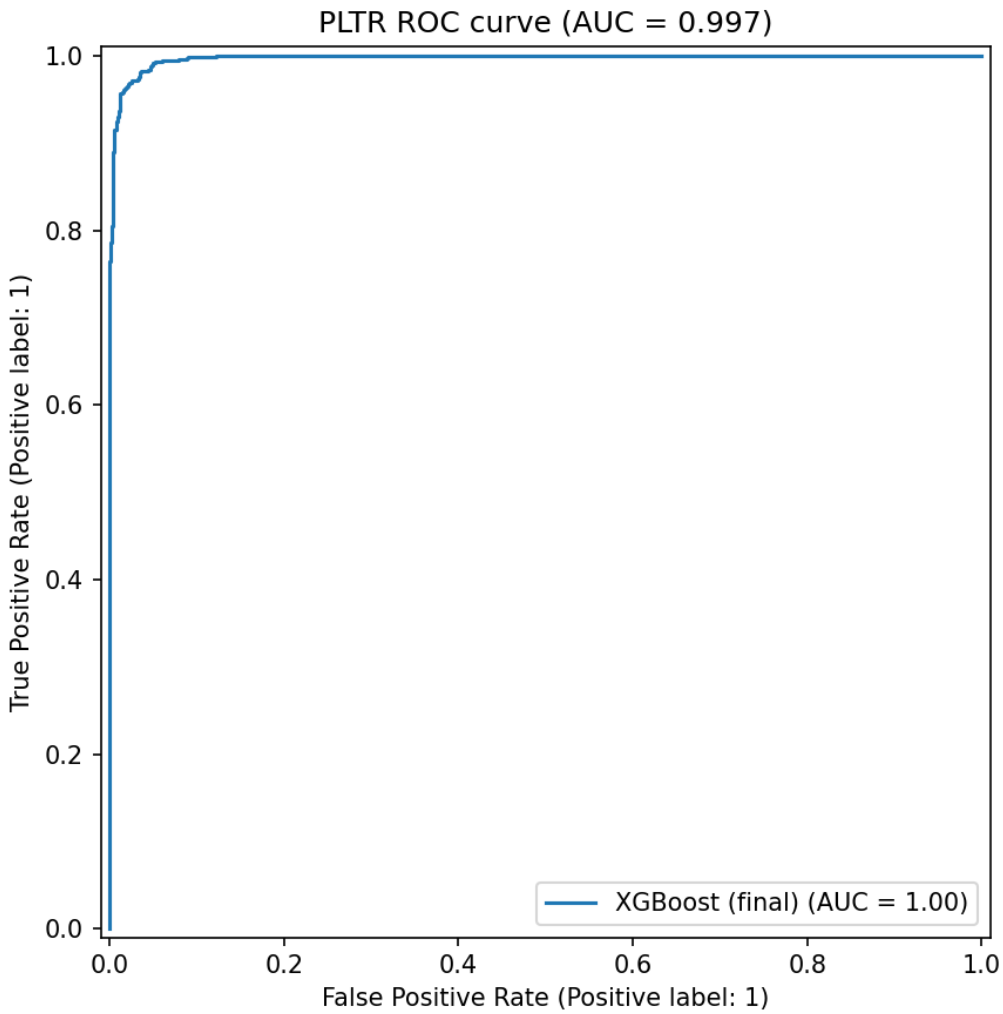

Figure 2. In-sample ROC curve for the tuned XGBoost model. The very high AUC (0.997) reflects training-set fit, not held-out performance.

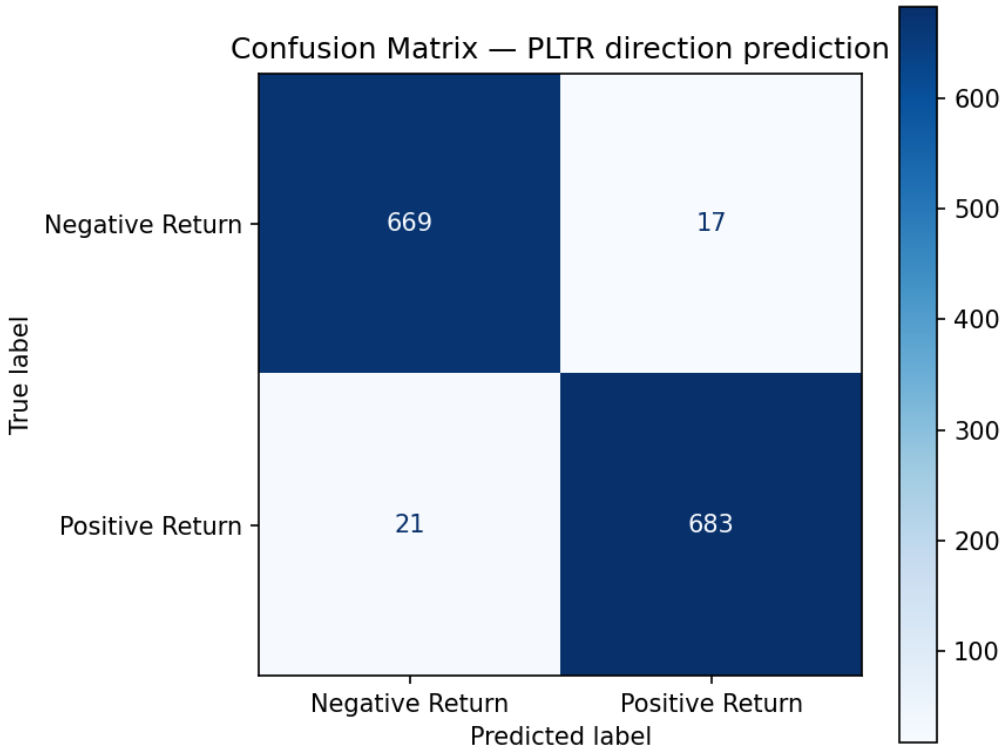

Figure 3. In-sample confusion matrix for the tuned XGBoost model.

Per-class metrics (in-sample classification report):

- Negative return: precision 0.97, recall 0.98, F1 0.97 (n = 686)
- Positive return: precision 0.98, recall 0.97, F1 0.97 (n = 704)

The contrast between the cross-validated accuracy (~0.52) and the in-sample AUC (0.997) is the headline finding of this study. With a max_depth of 6 and 846 trees, XGBoost can memorize 1,390 standardized feature vectors essentially perfectly even when the features carry almost no genuine signal about the target. This is a textbook example of why time-series cross-validation matters more than training-set metrics when assessing a financial forecasting model.

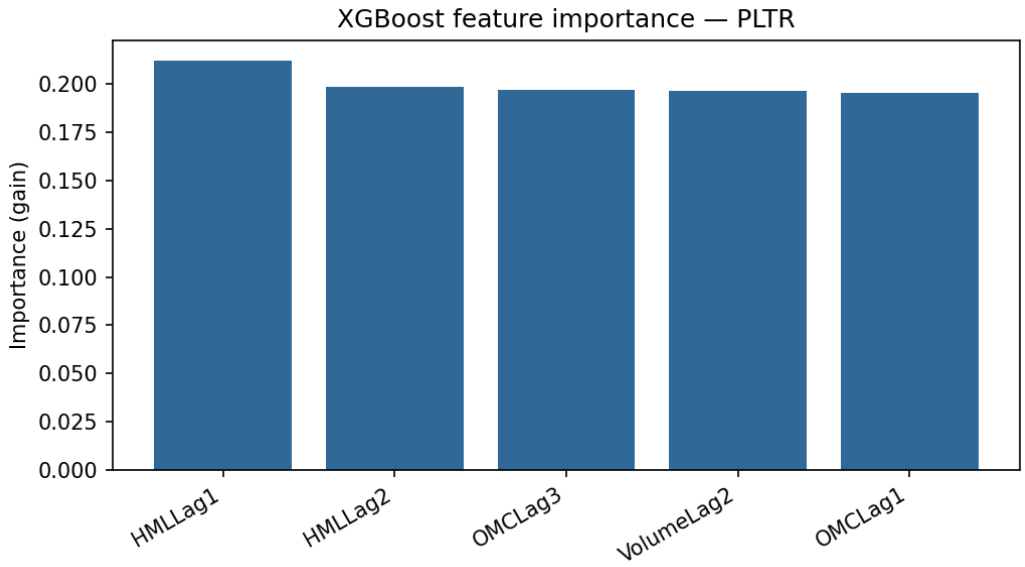

Figure 4. XGBoost gain-based feature importances on the final model.

5. Conclusions and Next Steps

Three takeaways from applying the jump-start methodology to PLTR:

- Feature selection is asset-specific. PLTR's AIC-best subset is built around intraday range (HMLLag1, HMLLag2) rather than lagged closes and long EMAs. Practitioners should not assume that a feature subset tuned on one asset transfers to another.
- Honest out-of-sample performance is roughly chance. Time-series cross-validated accuracy hovers near 52%, only marginally above the majority-class baseline. The training-set AUC of 0.997 is misleading and demonstrates the importance of disciplined evaluation.
- PLTR's short trading history limits what can be learned. With ~1,400 trading days versus WTI's 5,000+, both feature selection and hyperparameter tuning sit on a smaller statistical foundation.

Plausible next steps include:

- Adding non-price features that may actually carry directional information for PLTR specifically — sector ETF returns (e.g. XLK), volatility-index levels (VIX), Treasury-yield changes, or sentiment scores from news headlines and earnings-call transcripts.
- Redefining the target as a three-level outcome (clear up / stable / clear down) tied to a magnitude threshold, so the model can decline to trade on small moves rather than being forced into a binary call.
- Holding out a true forward test set (e.g. the most recent 6–12 months) and reporting metrics there, rather than on the training data, before treating any model output as actionable.

- Repeating the workflow on related tickers (e.g. C3.AI, Snowflake, Datadog) to test whether the HML-dominant feature pattern generalizes across AI/data-platform names.

6. References

- Miller, T. W. (2025). 451 Feature Engineering: Programming Assignment 1 — Technical Report. Northwestern University, MSDS 451.
- Akaike, H. (1974). A new look at the statistical model identification. IEEE Transactions on Automatic Control, 19(6), 716–723.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of KDD '16, 785–794.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825–2830.